



**VietNam National University
University of Engineering and Technology**

EMBEDDED SYSTEM FUNDAMENTALS

(ELT3240, NHẬP MÔN HỆ THỐNG NHÚNG)

Dr. Nguyễn Kiêm Hùng

Email: kiemhung@vnu.edu.vn



**Introduction to
Embedded Systems**

**Week
1-2**

**Introduction to C
You Are
Here.**

**Week
3**

**CPU:
ARM Cortex-M**

**Week
4**

**Memory
and Interfaces**

**Week
5-6**

**ARM-based
Embedded System**

**Week
7**

Embedded Software

**Week
8-9**

**Real-time
Operating systems**

**Week
10-12**

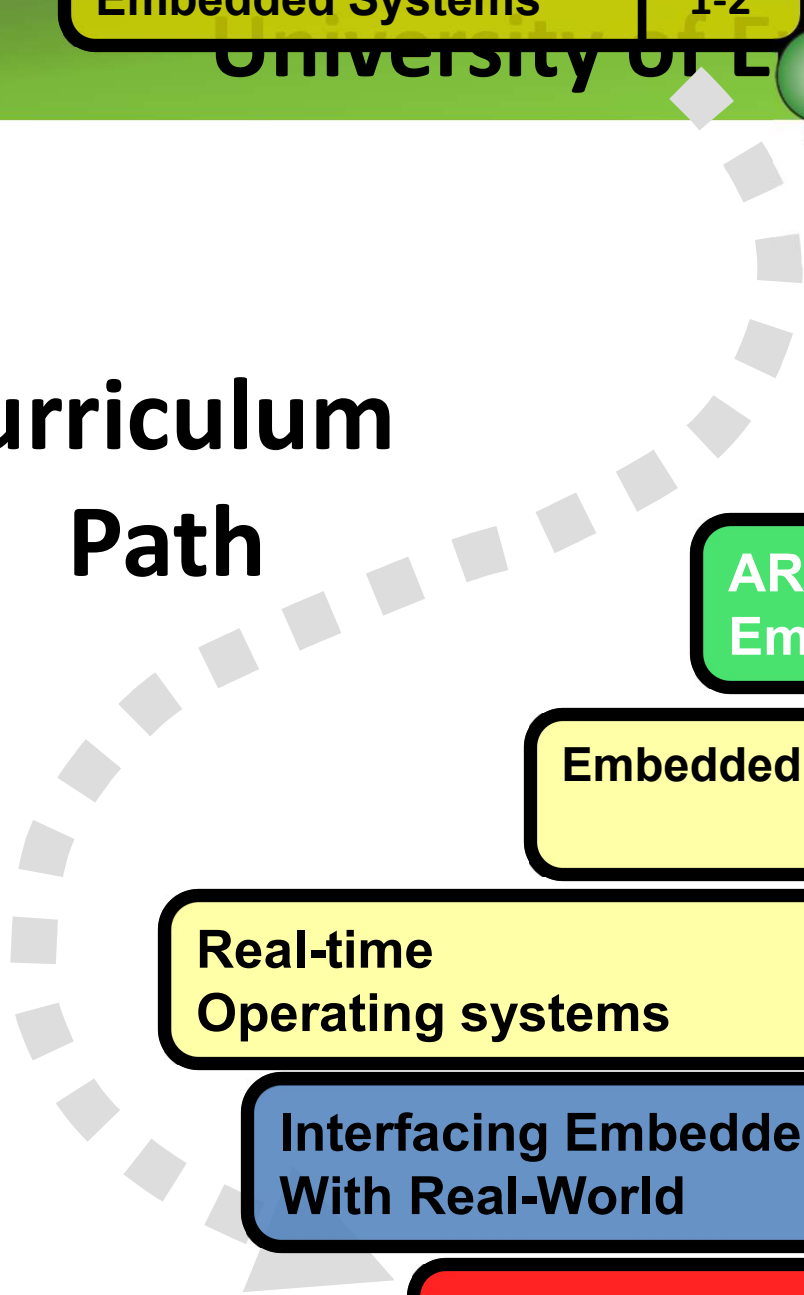
**Interfacing Embedded
With Real-World**

**Week
13-14**

Project

**Week
15**

**Curriculum
Path**



Objectives

In this lecture you will be introduced to:

- A brief review of computer architecture taxonomy
- ARM instruction set architecture

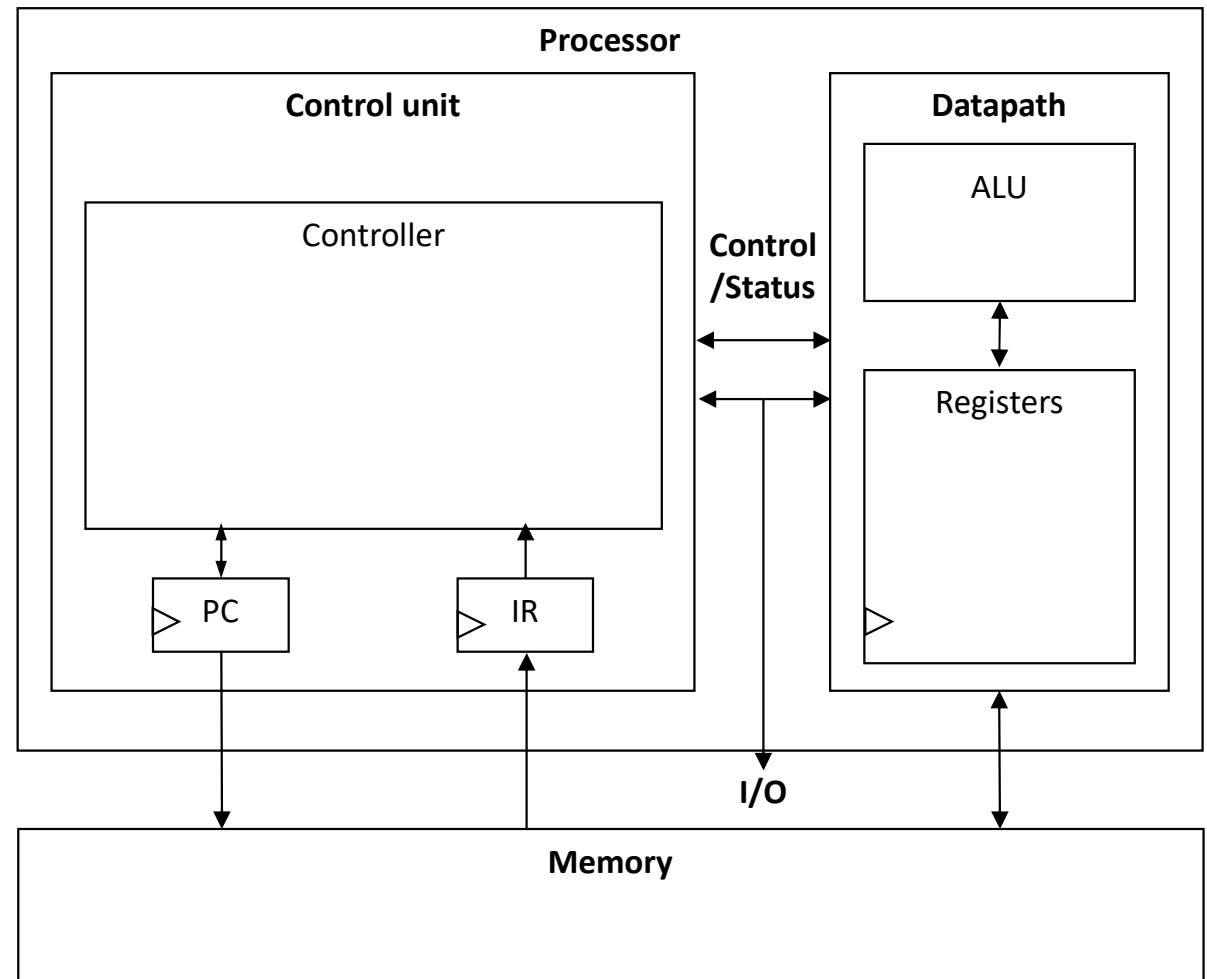
Outline



- **Basic Processor Architecture**
- Computer architecture taxonomy
- Basic concept of Instruction set
- ARM instruction set
 - *Case study*: ARM Cortex-M architecture
- Summary

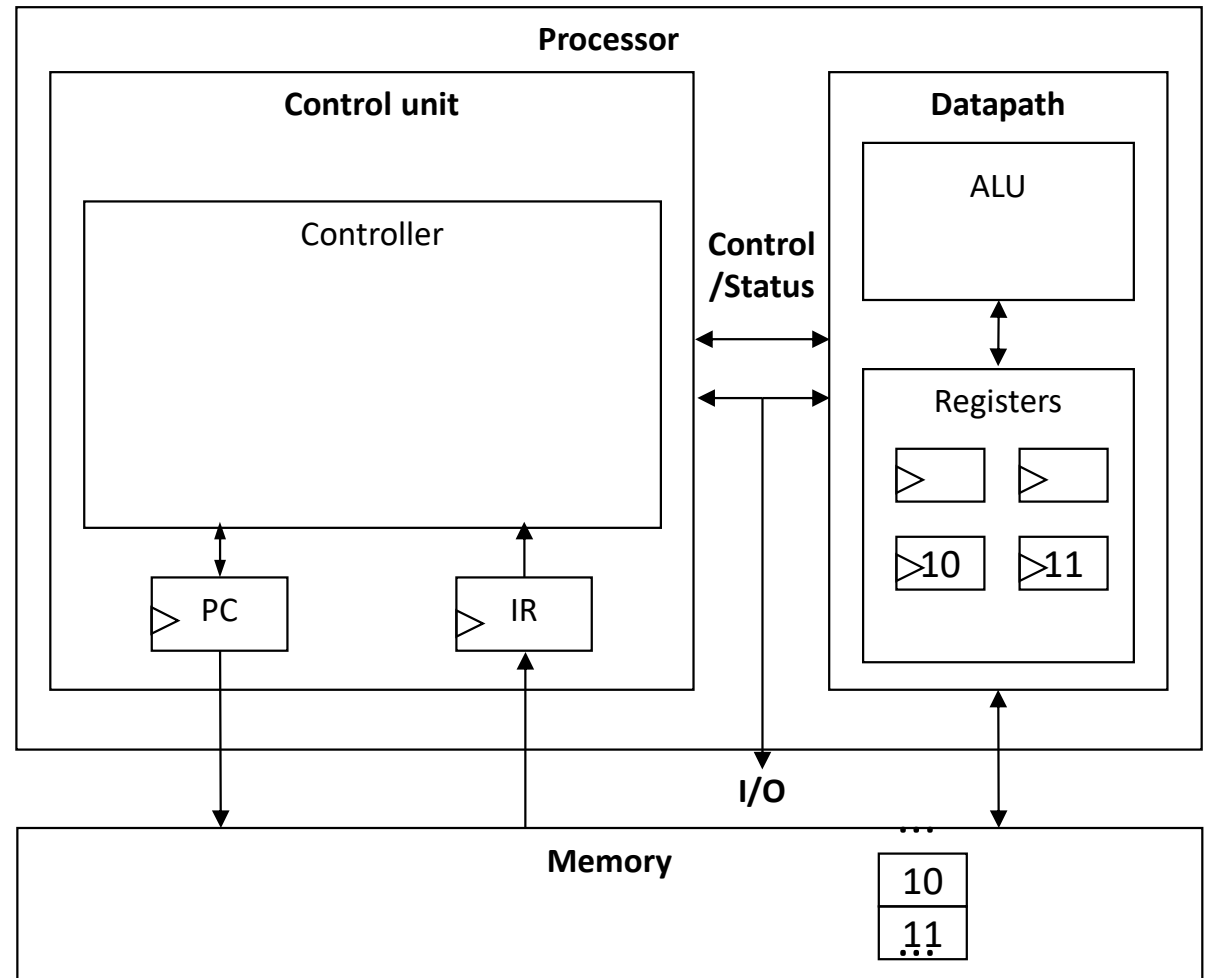
Basic Processor Architecture

- **Processor = Central processing Unit (CPU)**
 - CPU fetches instructions from memory.
 - Separating CPU and memory that distinguishes **programmable computer** from a general **finite-state machine**.
 - CPU registers: **program counter (PC)**, **instruction register (IR)**, general-purpose registers, etc.



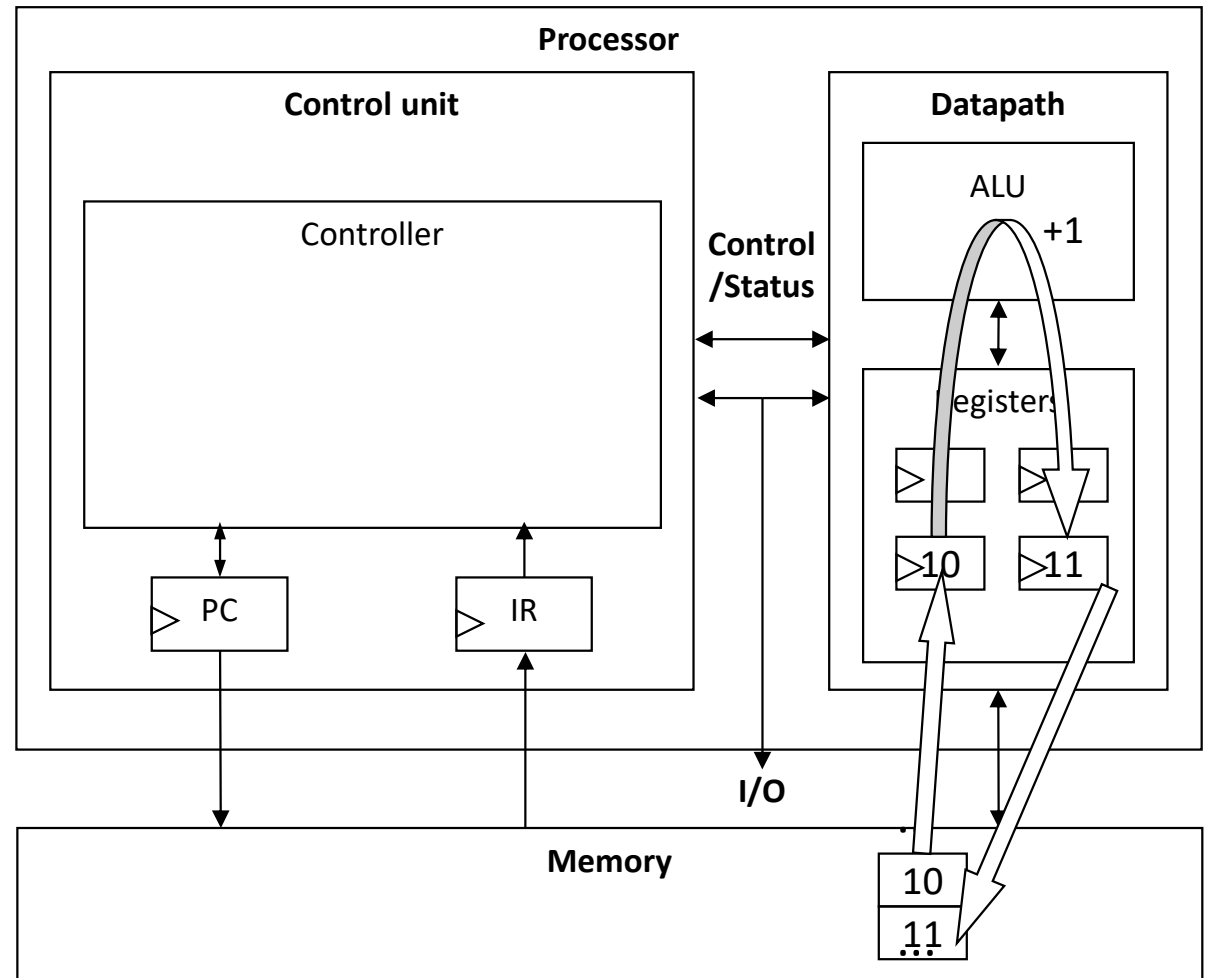
Datapath

- **Function**
 - transforming data and storing temporary data
- **Structure:**
 - ALU (arithmetic-logic unit): transforming data through operations
 - Registers: storing temporary data
 - internal data bus



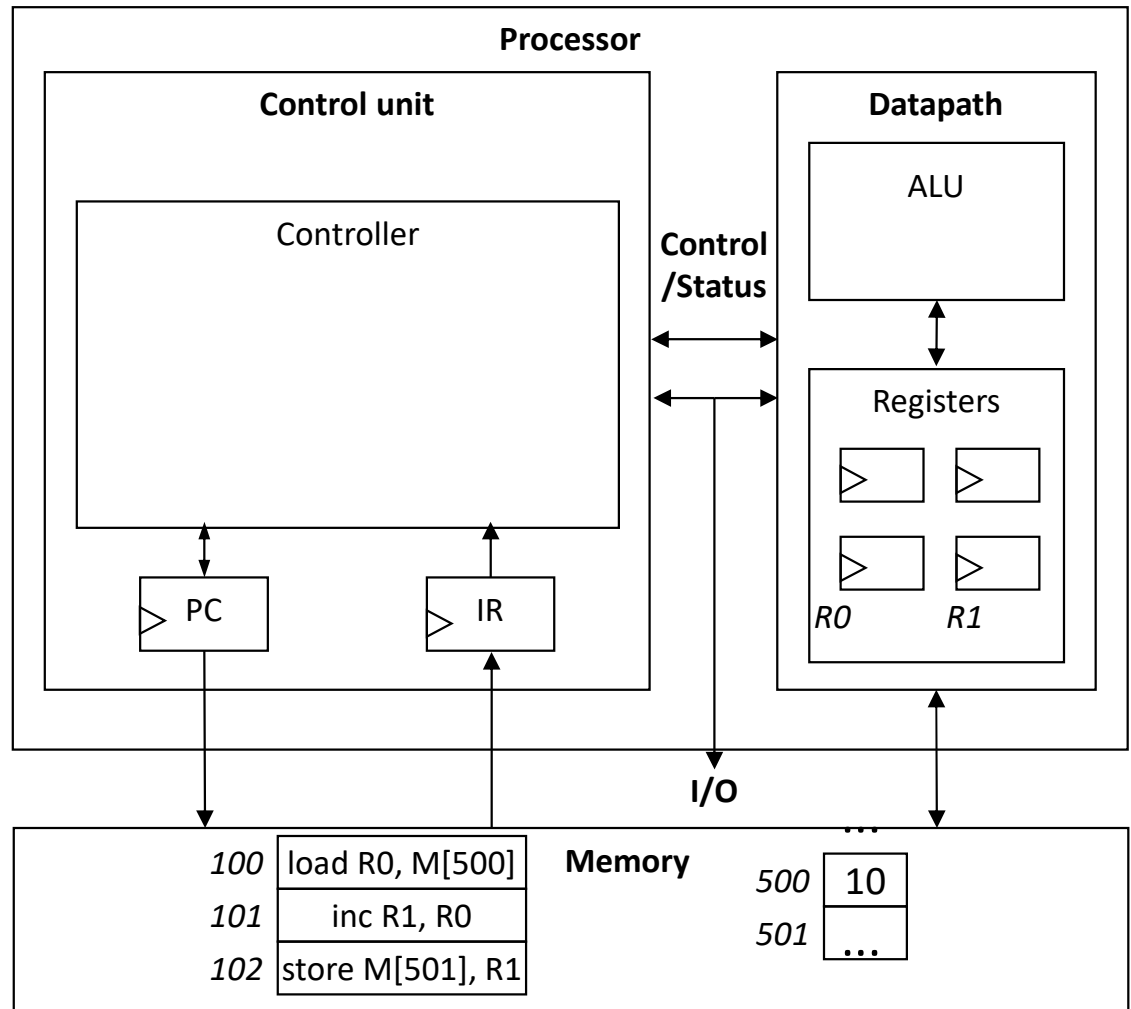
Datapath Operations

- Load
 - Read memory location into register
- ALU operation
 - Input certain registers through ALU, store back in register
- Store
 - Write register to memory location



Control Unit

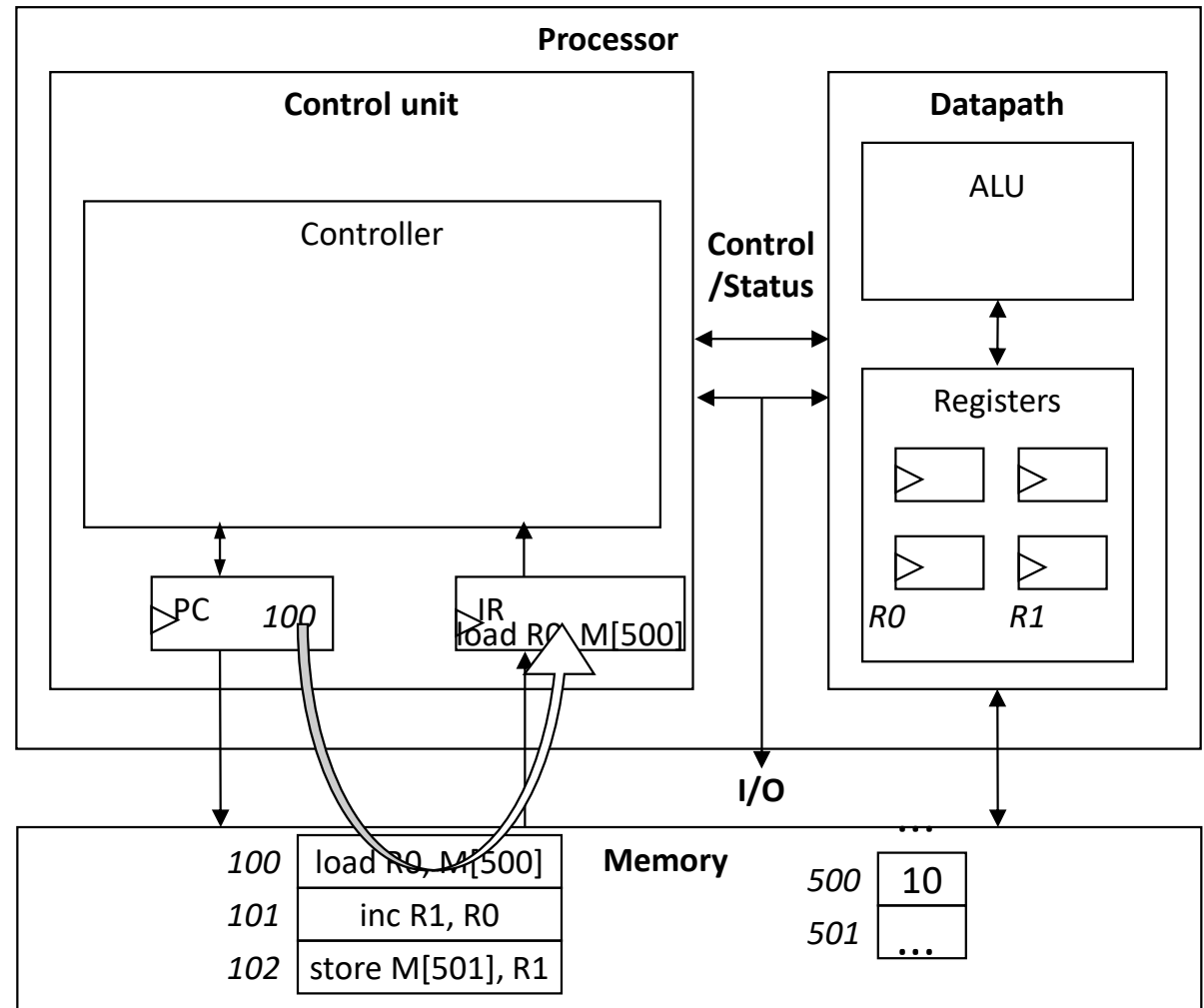
- **Control unit:** configures the datapath operations
 - Sequence of desired operations (“instructions”) stored in memory – “**Program**”
- **Instruction cycle** – broken into several sub-operations, each one clock cycle, e.g.:
 - *Fetch*: Get next instruction into IR
 - *Decode*: Determine what the instruction means
 - *Fetch operands*: Move data from memory to datapath register
 - *Execute*: Move data through the ALU
 - *Store results*: Write data from register to memory



Control Unit Sub-Operations

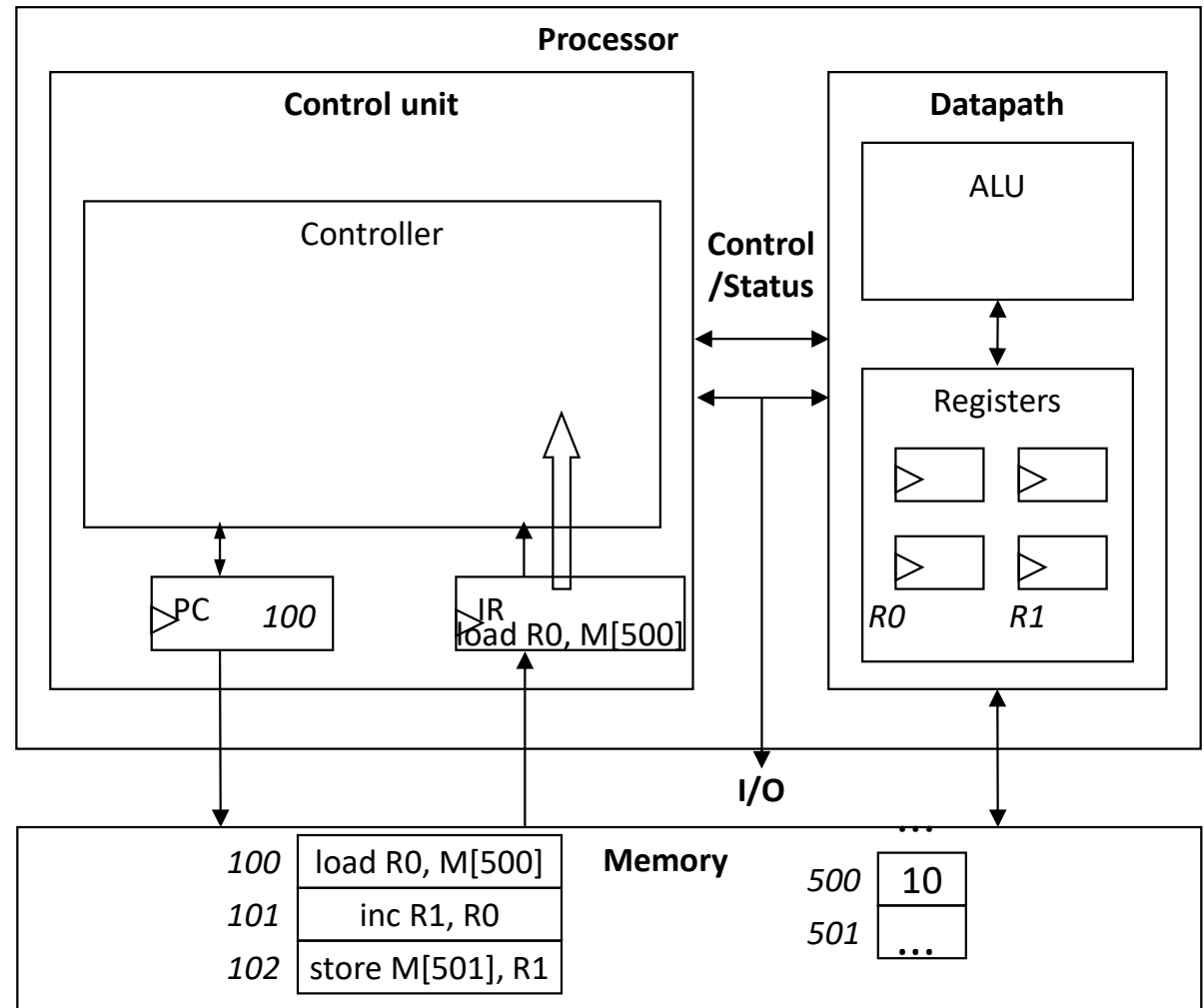
- **Fetch**

- Get next instruction into IR
- PC: program counter, always points to next instruction
- IR: Instructions Register, holds the fetched instruction



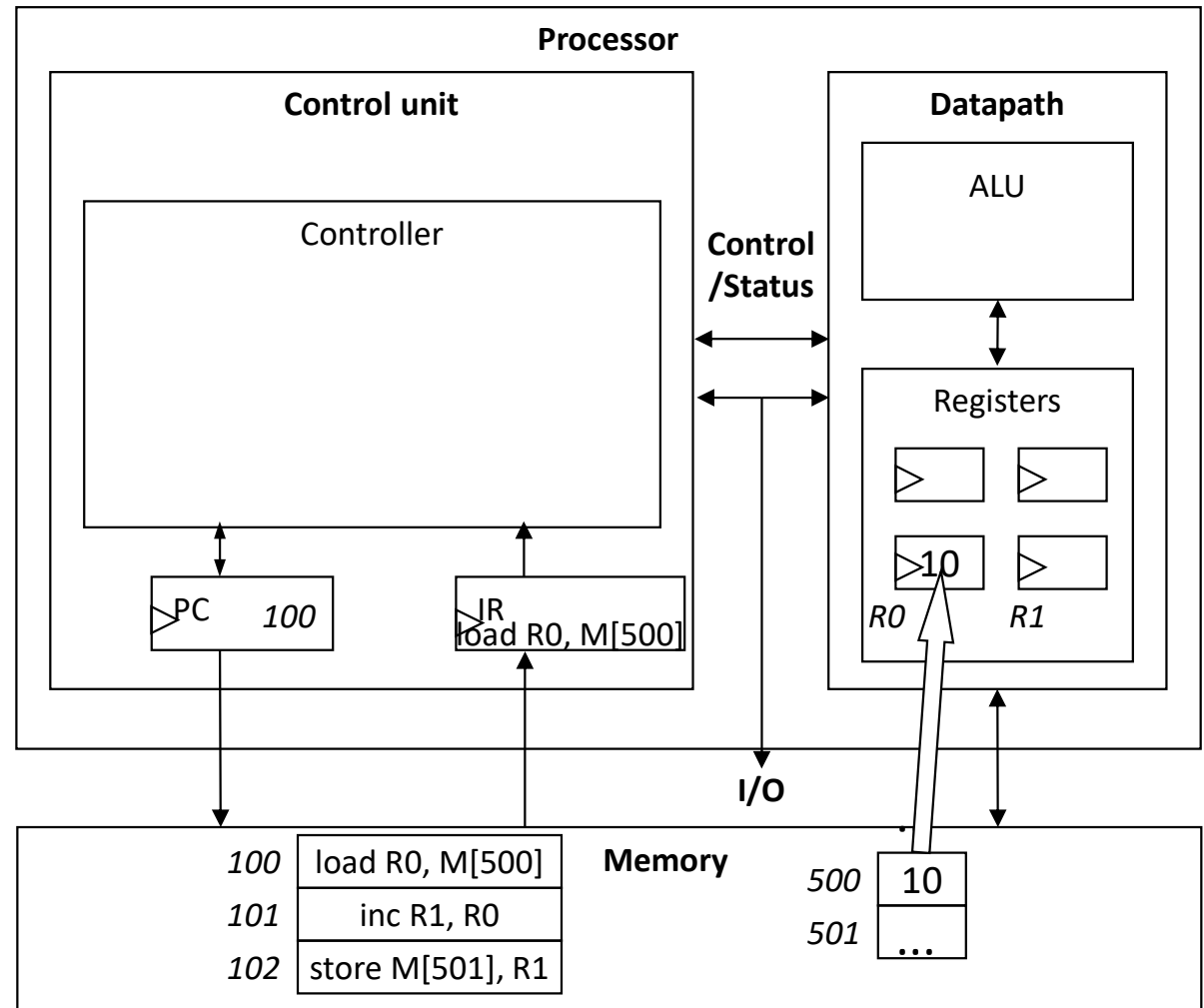
Control Unit Sub-Operations

- **Decode**
 - Determine what the instruction means



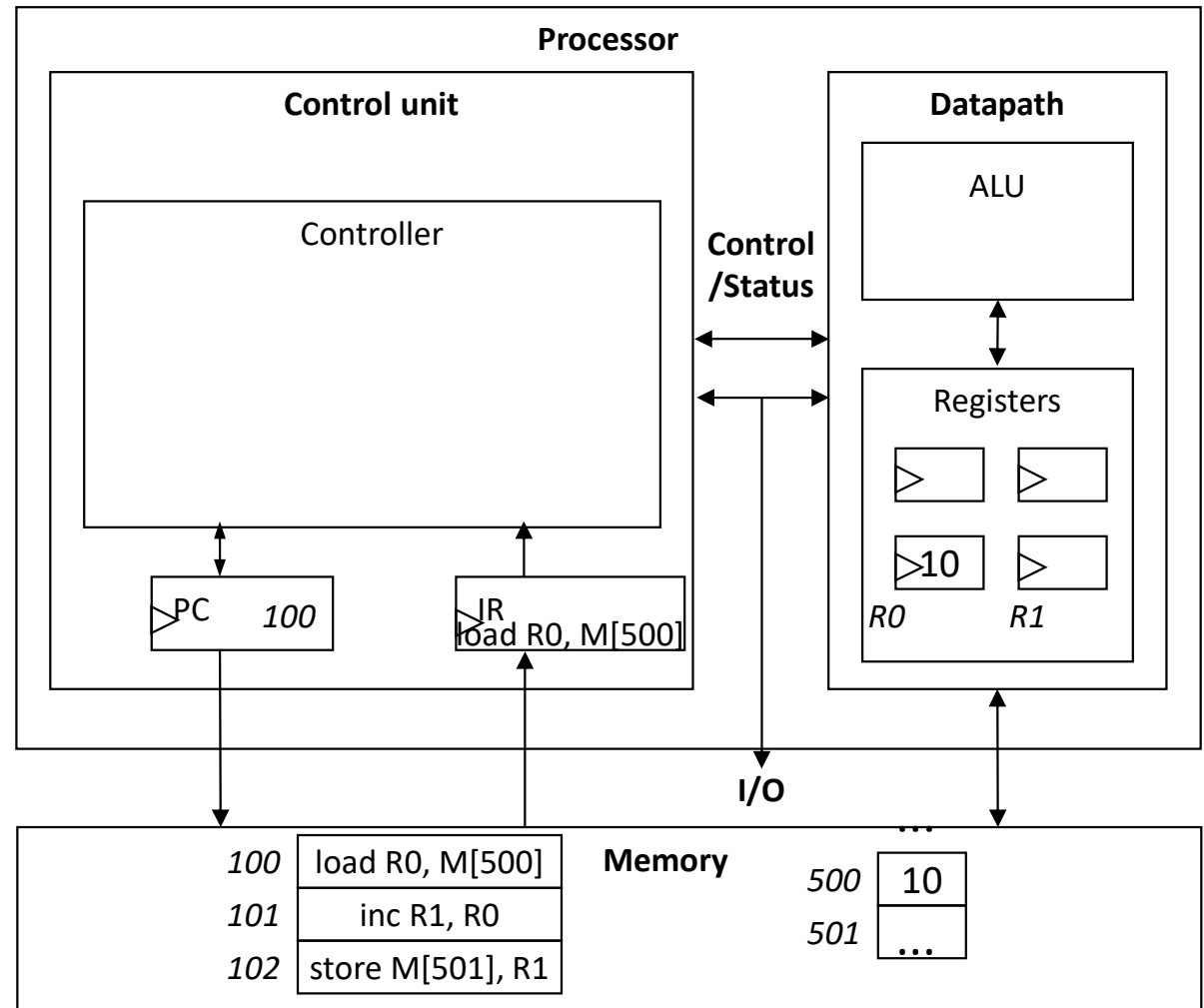
Control Unit Sub-Operations

- **Fetch operands**
 - Move data from memory to datapath register



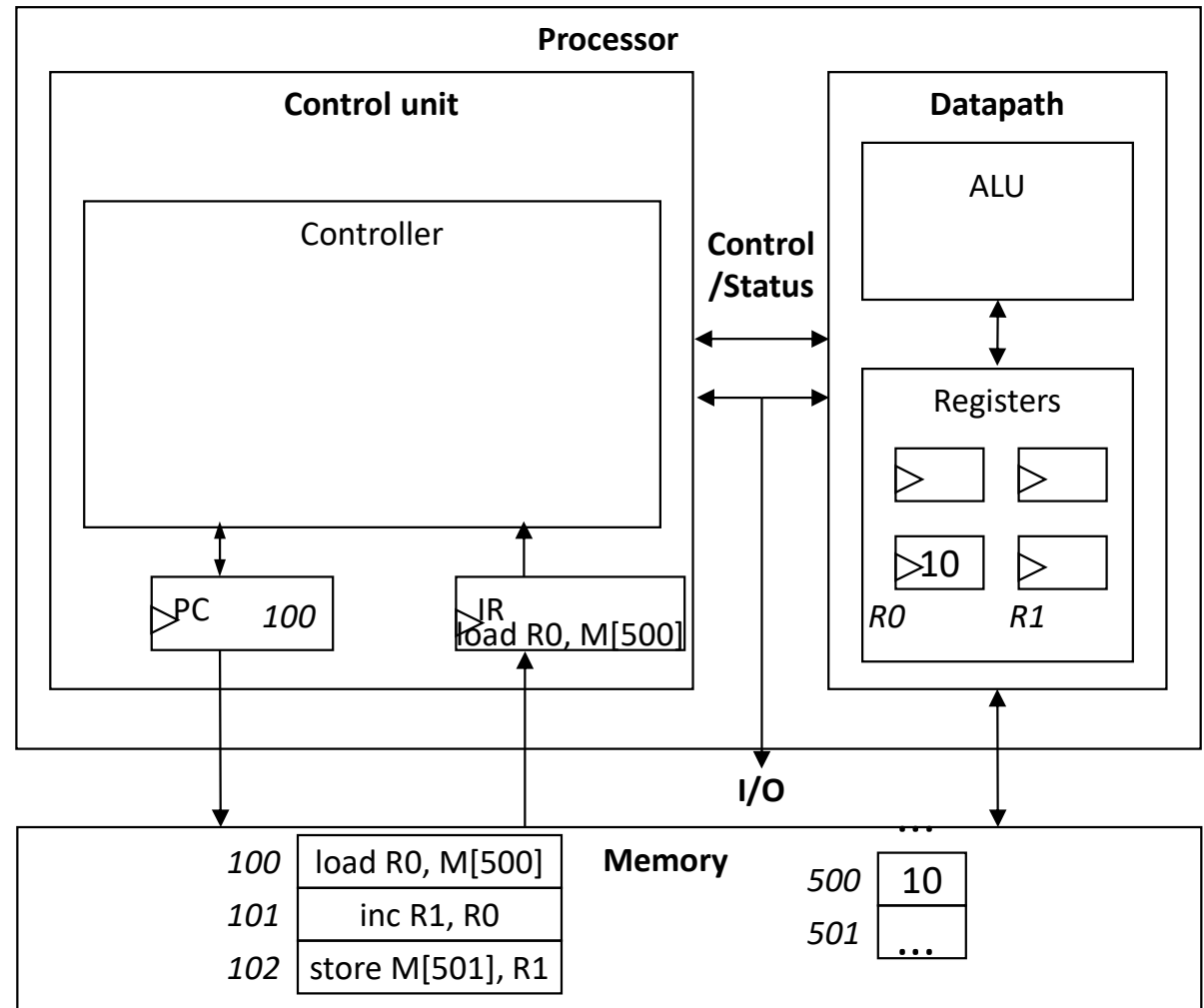
Control Unit Sub-Operations

- **Execute**
 - Move data through the ALU
 - This particular instruction does nothing during this sub-operation

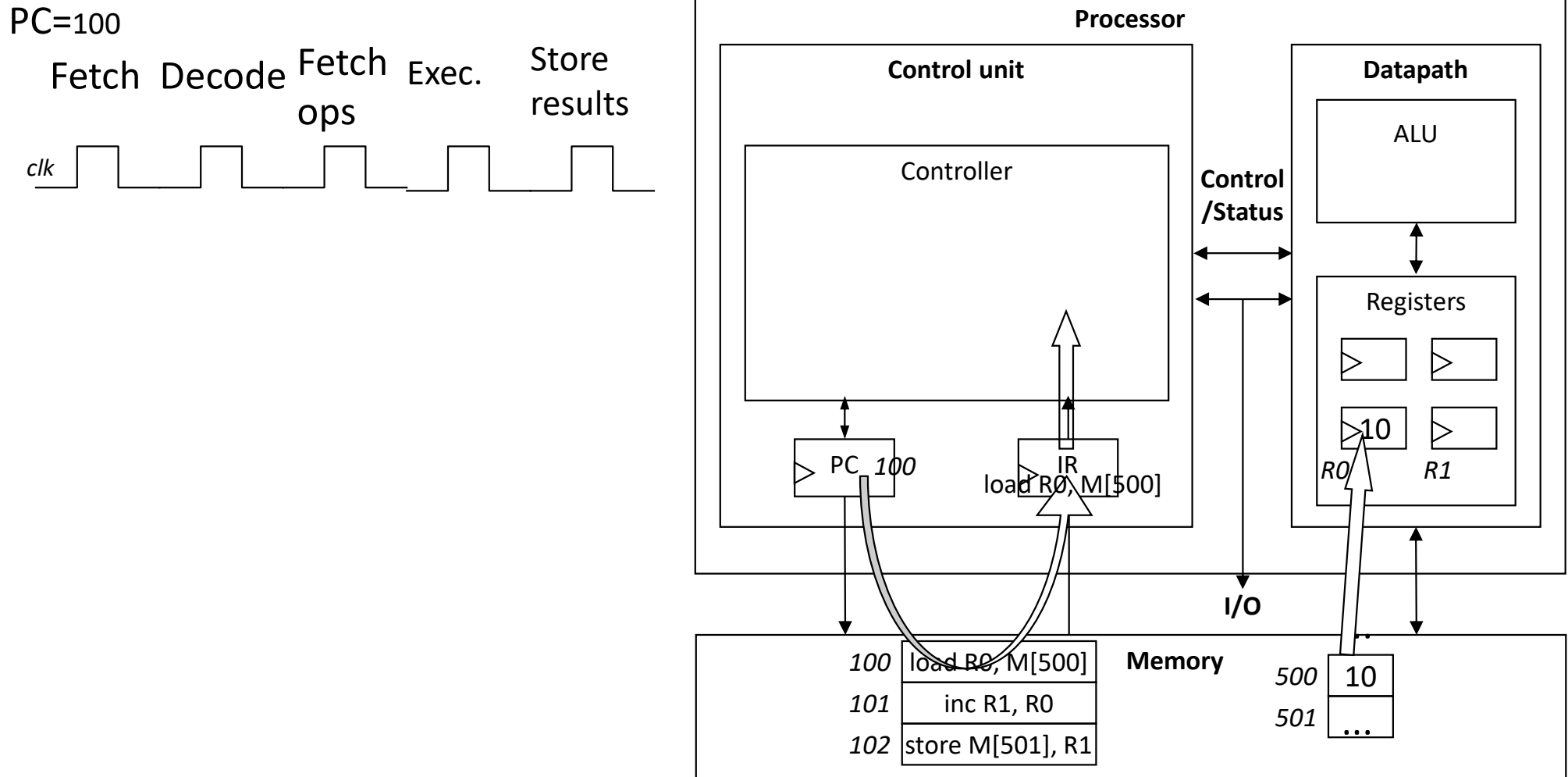


Control Unit Sub-Operations

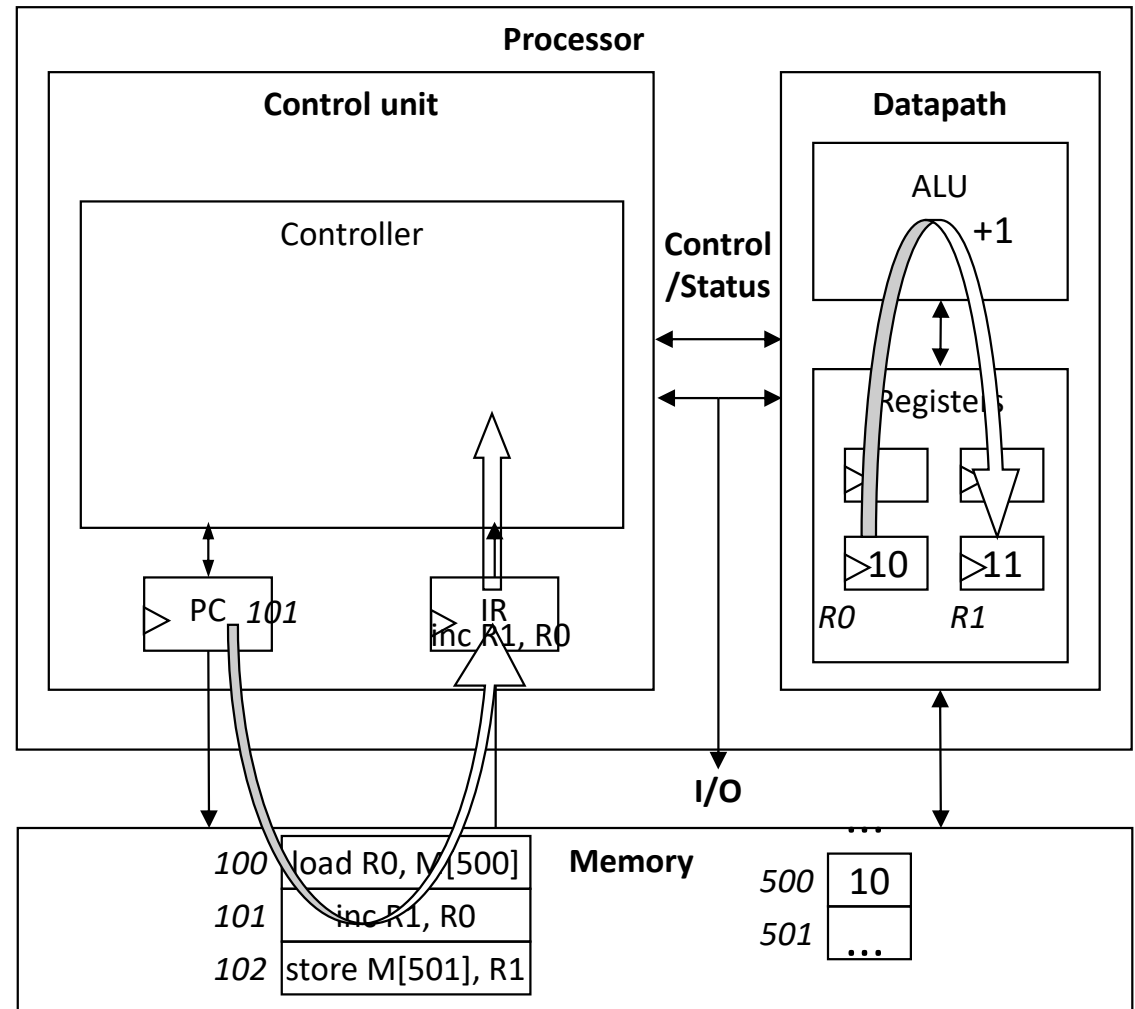
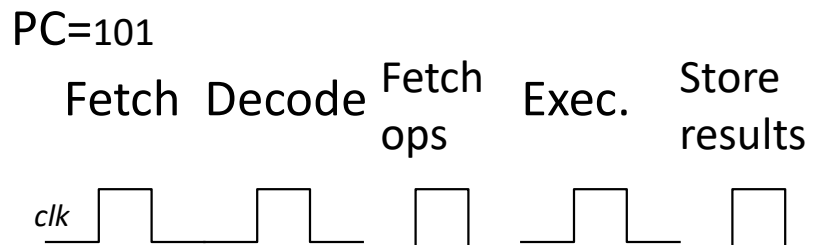
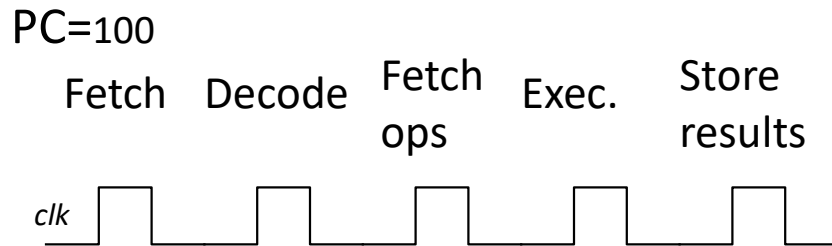
- **Store results**
 - Write data from register to memory
 - This particular instruction does nothing during this sub-operation



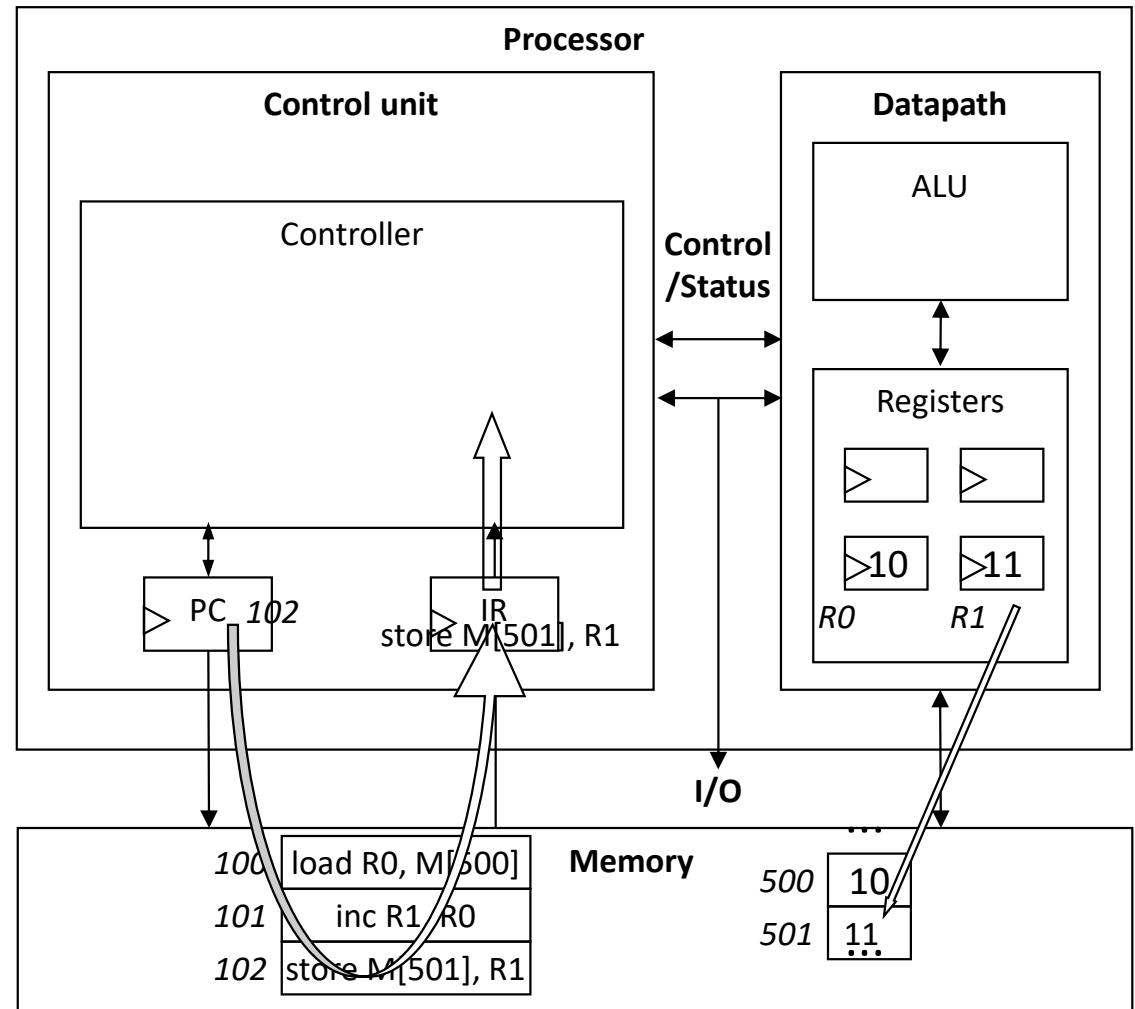
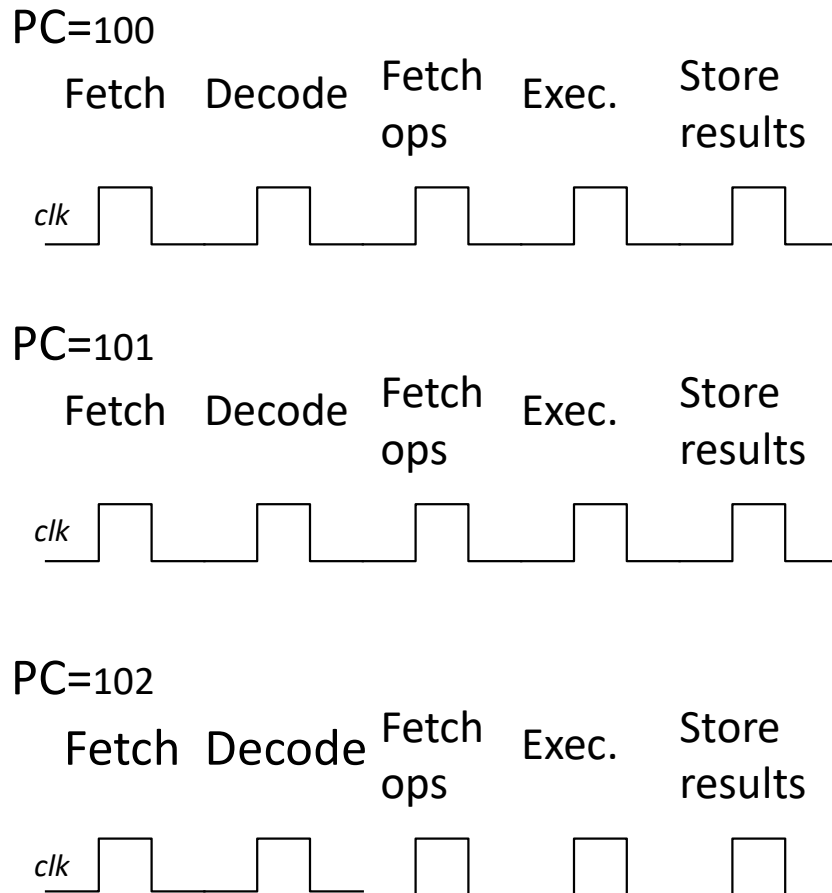
Instruction Cycles



Instruction Cycles



Instruction Cycles



Computer architecture taxonomy

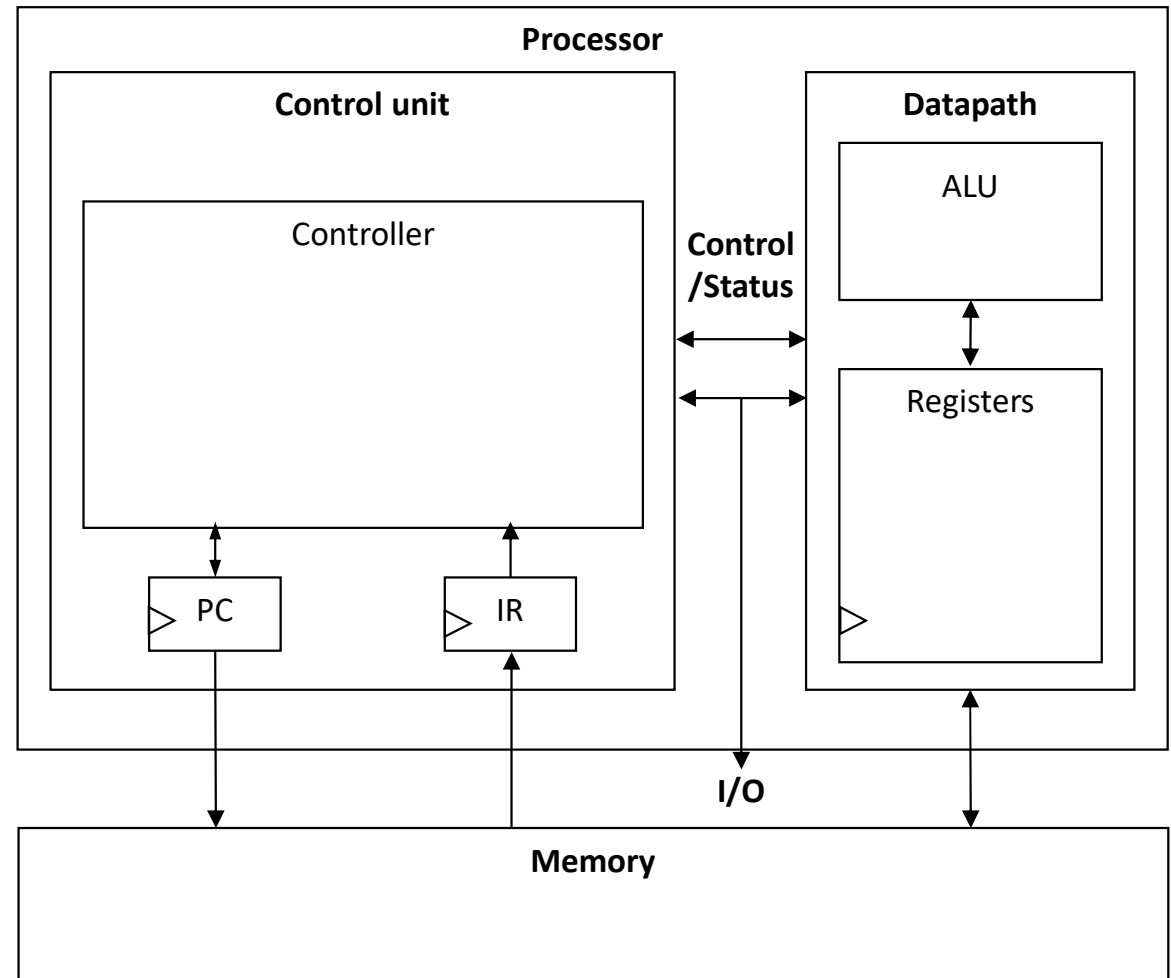
- **N-bit processor**



What does 8-bit, 16-bit, 32-bit etc. mean?

Architectural Considerations

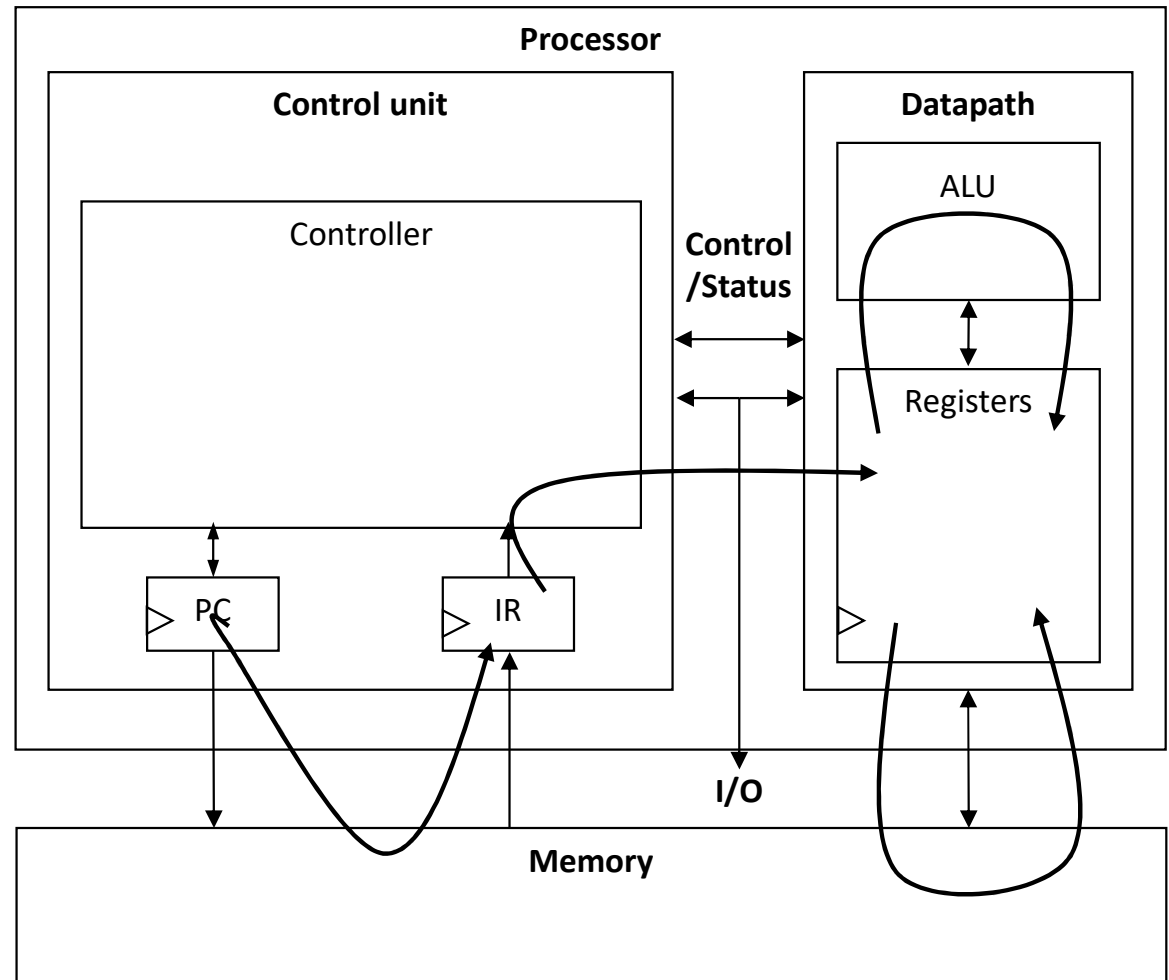
- ***N-bit* processor**
 - N-bit ALU, registers, buses, memory data interface
 - Embedded: 8-bit, 16-bit, 32-bit common
 - Desktop/servers: 32-bit, even 64
- PC size determines address space



Architectural Considerations

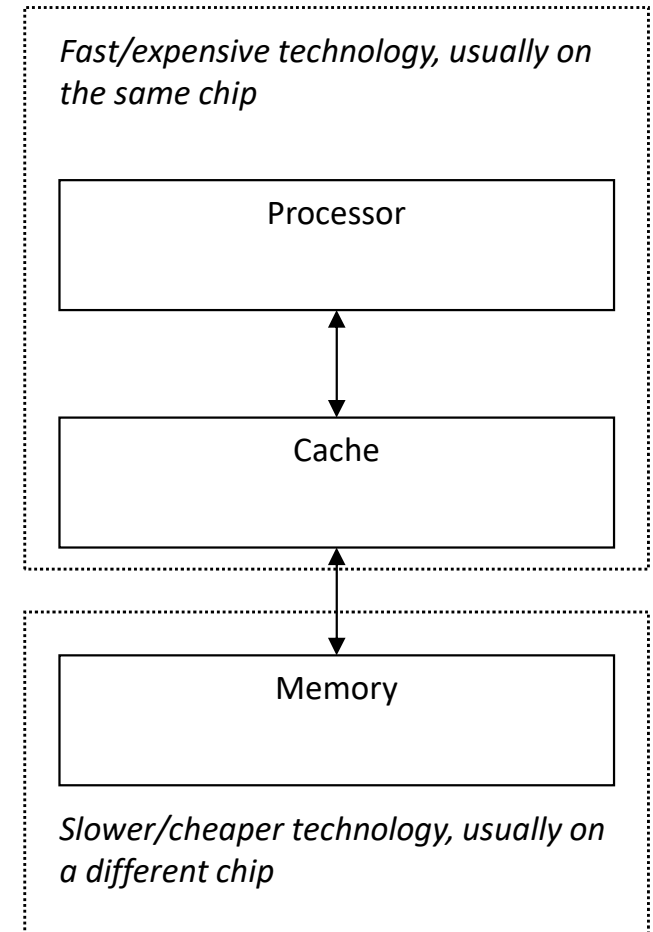
- **Clock frequency**

- Inverse of clock period
- Clock period must be longer than longest register-to-register delay in entire processor
- Memory access is often the longest



Cache Memory

- Memory access may be slow
- Cache is small but fast memory close to processor
 - Holds copy of part of memory
 - Hits and misses



Outline

- **Basic Processor Architecture**



- **Computer architecture taxonomy**

- Basic concept of Instruction set
- ARM instruction set
 - *Case study*: ARM Cortex-M architecture
- Summary

Computer architecture taxonomy

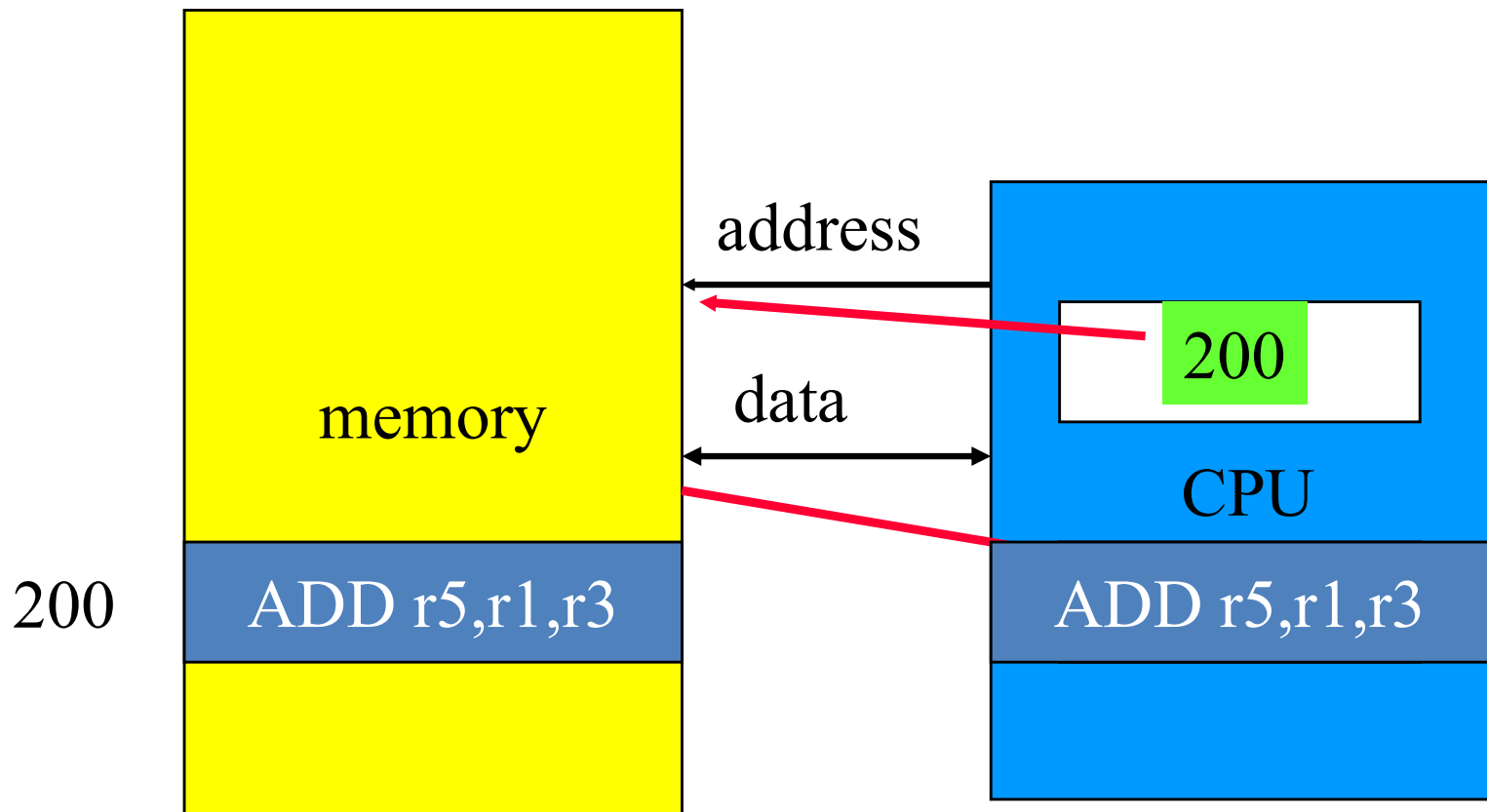
- **Von Neumann vs. Harvard architecture**



**What is difference between Von Neumann vs.
Harvard architecture?**

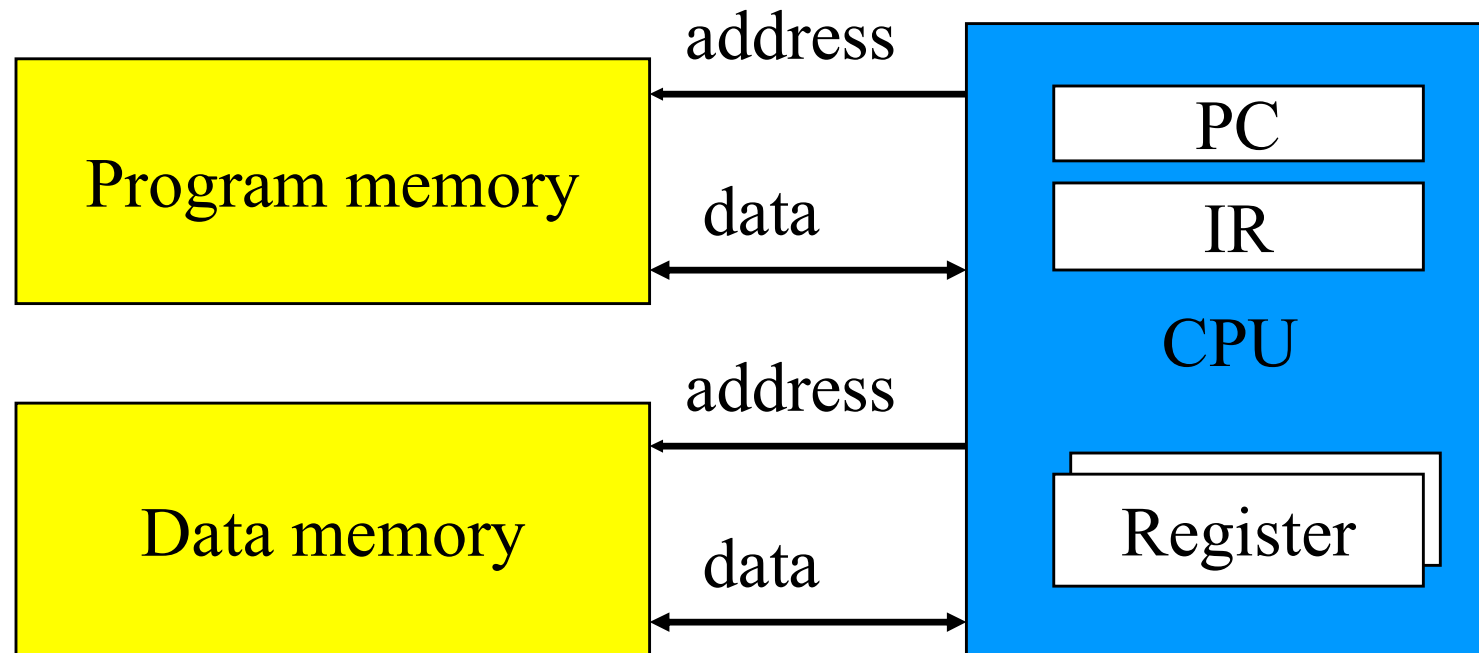
Computer architecture taxonomy

- **Von Neumann architecture**
 - Memory holds both data and instructions.



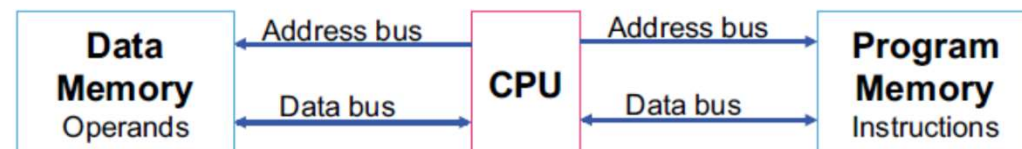
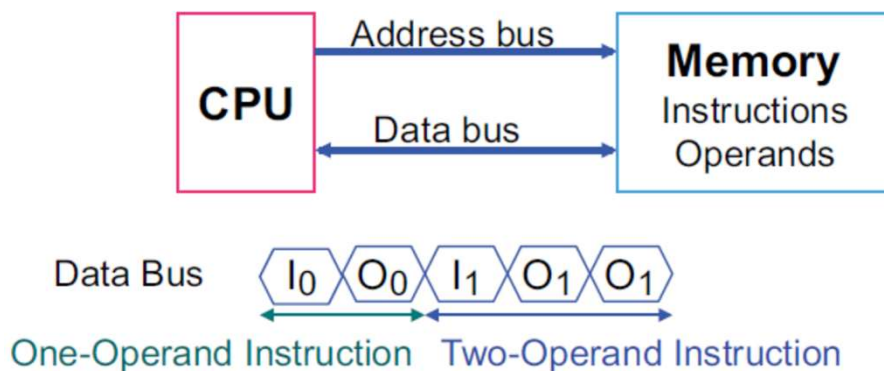
Computer architecture taxonomy

- **Harvard architecture**



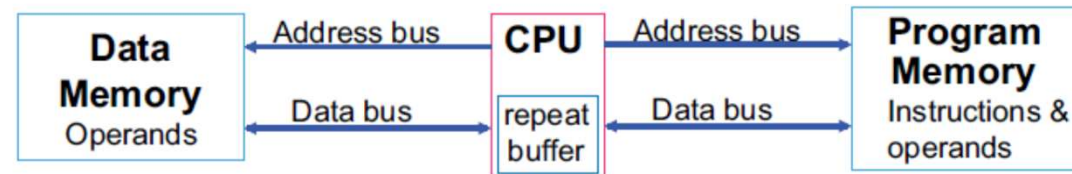
Computer architecture taxonomy

- **Von Neumann vs. Harvard architecture**
 - Harvard can't use self-modifying code.
 - Harvard allows two simultaneous memory fetches.
 - Most DSPs use Harvard architecture for streaming data:
 - greater memory bandwidth;
 - Higher performance.
 - Low-cost microprocessors use von Neumann architecture



Program Data Bus $I_0, I_1, I_2, \dots, I_3$

Data Data Bus O_0, O_1, O_{20}, O_{21}



Program Data Bus $I_0, I_1, O_{10}, O_{20}, O_{30}$

Data Data Bus $O_0, O_{11}, O_{21}, O_{31}$

Computer architecture taxonomy

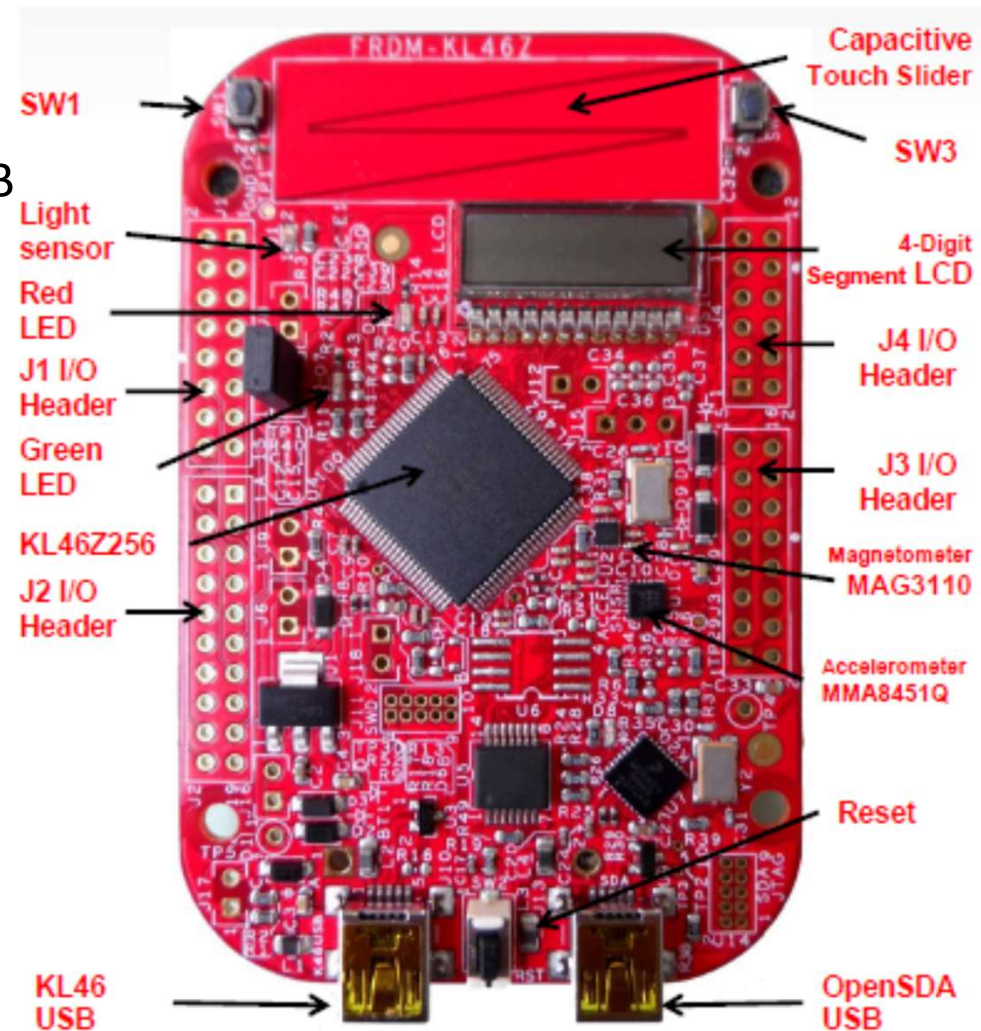
- **CISC vs. RISC**

CISC: Complex Instruction Set Computers	RISC: Reduced Instruction Set Computers
Many addressing modes; many operations.	Simpler instructions, faster execution speeds per instruction
Fewer instructions to execute a given task in comparison with RISC	-
Programs for CISC take less storage space than programs for RISC	Cheaper to implement
Arithmetic or other instructions may read their operand from memory and could write the result in memory	Load/Store architecture – only load and store are used to access the external memory

Computer architecture taxonomy

- **FRDM-KL46Z Kit:**

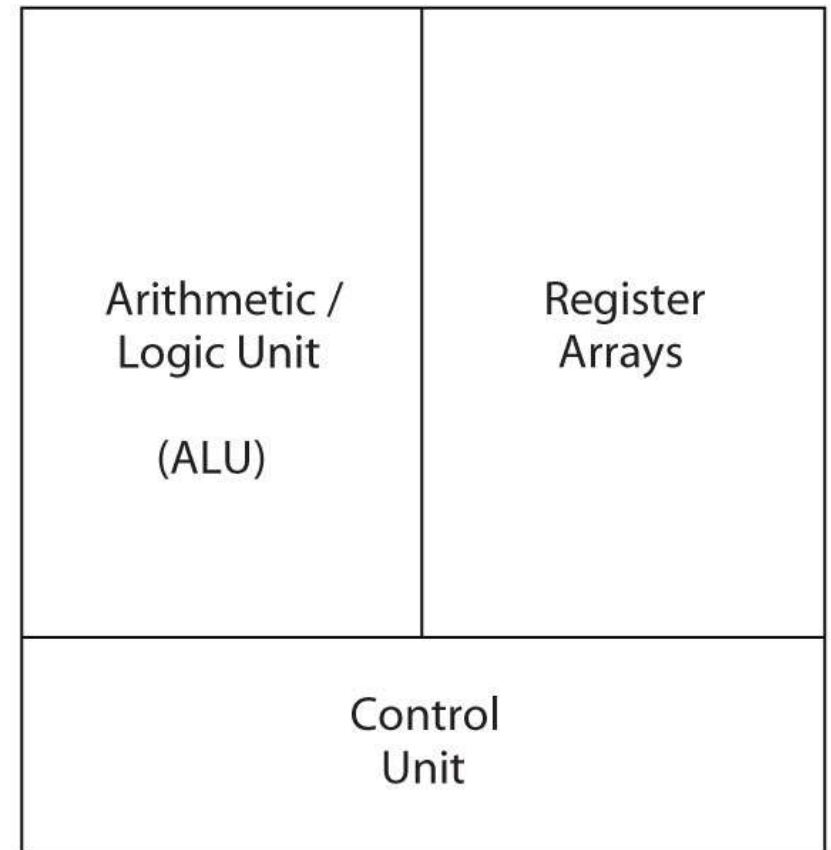
- MKL46Z256VLLZ4 ARM Cortex™-M0+ **Processor-based MCU** (48 MHz, 256KB Flash, 32 KB RAM, Low power, 100LQFP package)
- Dual role USB interface with mini-B USB connector
- 4 digit segment LCD module
- Capacitive touch slider
- Ambient light sensor
- MMA8451Q accelerometer
- MAG3110 Magnetometer
- 2 user LEDs
- 2 user push buttons
- Programmable OpenSDA debug interface



Computer architecture taxonomy

- **Microprocessor** vs. **Microcontroller**

- A microprocessor incorporates the functions of a computer's central processing unit (CPU) on a single integrated circuit (IC)
- **Microprocessor** is a multipurpose, programmable device that accepts digital data as input, processes it according to instructions stored in its memory, and provides results as output



Computer architecture taxonomy

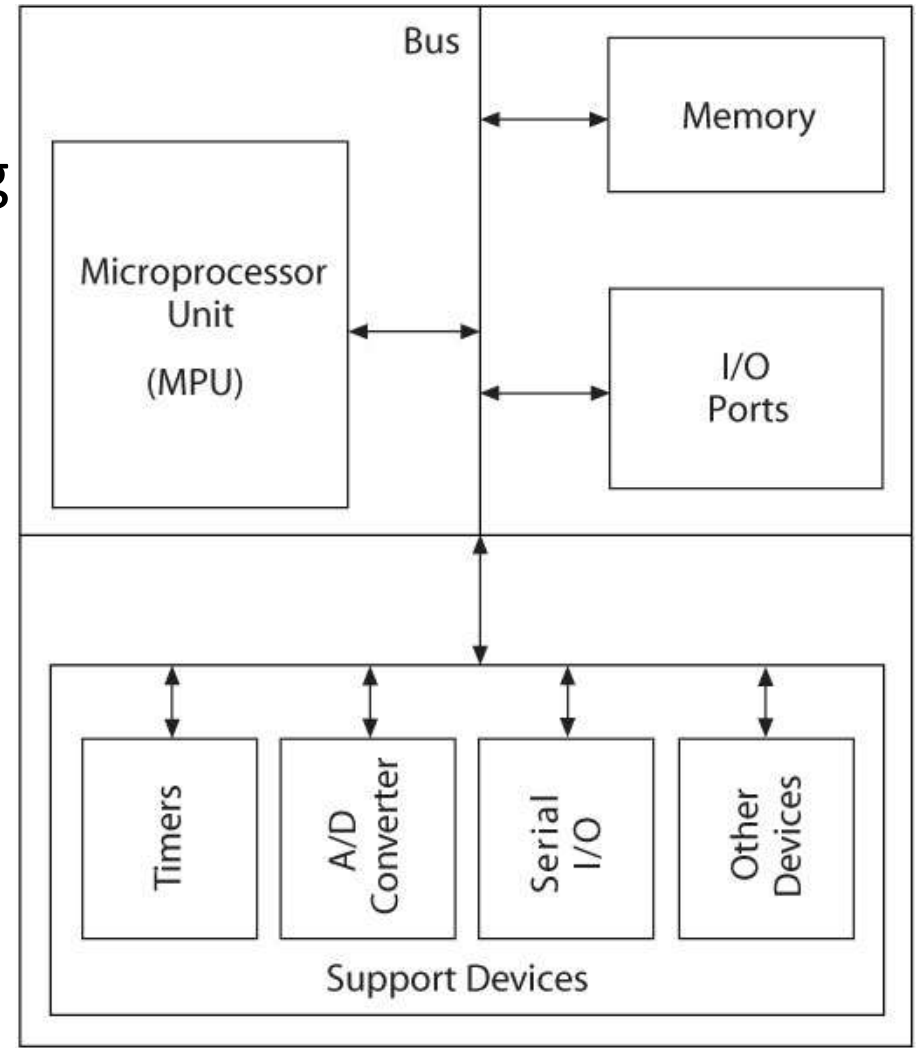
- **Microprocessor** vs. **Microcontroller**

Microprocessor	CPU MIPS at MHz	Year released
<u>Intel 8080</u>	640 kIPS / 2 MHz	1974
<u>Motorola 68000</u>	1 MIPS / 8 MHz	1979
<u>Intel 286</u>	2.66 MIPS / 12 MHz	1982
<u>Motorola 68020</u>	4 MIPS / 20 MHz	1984
<u>ARM2</u>	4 MIPS / 8 MHz	1986
<u>Motorola 68030</u>	11 MIPS / 33 MHz	1987
<u>Intel 386DX</u>	8.5 MIPS / 25 MHz	1988
<u>Motorola 68040</u>	44 MIPS / 40 MHz	1990
<u>Intel 486DX</u>	54 MIPS / 66 MHz	1992
<u>PowerPC 600s (G2)</u>	35 MIPS / 33 MHz	1994
<u>Motorola 68060</u>	88 MIPS / 66 MHz	1994
<u>Intel Pentium Pro</u>	541 MIPS / 200 MHz	1996
<u>ARM 7500FE</u>	35.9 MIPS / 40 MHz	1996
<u>PowerPC G3</u>	525 MIPS / 233 MHz	1997
<u>Zilog eZ80</u>	80 MIPS / 50 MHz	1999
<u>Intel Pentium III</u>	1,354 MIPS / 500 MHz	1999
<u>AMD Athlon</u>	3,561 MIPS / 1.2 MHz	2000
<u>AMD Athlon XP 2400+</u>	5,935 MIPS / 2.0 GHz	2002
<u>Pentium 4 Extreme</u>	9,726 MIPS / 3.2 GHz	2003
<u>AMD Athlon 64 3800+ X2 (Dual Core)</u>	14,564 MIPS / 2.0 GHz	2005
<u>Intel Core 2 X6800</u>	27,079 MIPS / 2.93 GHz	2006
<u>Intel Core 2 Extreme QX9770</u>	59,455 MIPS / 3.2 GHz	2008

Computer architecture taxonomy

- **Microprocessor vs. Microcontroller**

- **Microcontroller (MCU)** is integrated electronic computing device that includes three major components on a single chip:
 - Microprocessor
 - Memory
 - I/O devices
- **A microcontroller is one form of a single-chip computer**



Computer architecture taxonomy

- **Microprocessor vs. Microcontroller**
 - **Examples:**
 - **Intel 8051 microcontroller:**
 - 8-bit microprocessor
 - 1MIPS/12MHz
 - small on-chip RAM of 128 bytes for data and 4K on-chip ROM for code.
 - 4 bi-directional I/O ports,
 - two counter/timers, UART...
 - **Motorola 68HC16:**
 - 16-bit microcontroller
 - 768 bytes on-chip RAM and 64K extended RAM.
 - **Microchip's PIC; ATMEL's AT89, AVR, ATmega, ...**

Computer architecture taxonomy

- **Microprocessor vs. Microcontroller**

Microprocessor	Microcontroller
A microprocessor is a single-chip CPU. In addition to the memory, we also need I/O devices to build a useful system.	A microcontroller is one form of a single-chip computer that includes a processor, memory, and I/O devices.
Microprocessor are designed to perform unspecific tasks - the relationship of input and output is not defined. - high amount of resources like RAM, ROM, I/O ports	Microcontrollers are designed to perform specific tasks. - the relationship of input and output is defined. - small resources like RAM, ROM, I/O ports

The clock speed of the Microprocessor is quite high as compared to the microcontroller.

- Microcontrollers operate from a few MHz to 30 to 50 MHz,
- Microprocessors can operate up to 5GHz

Computer architecture taxonomy

- **Little Endian vs. Big Endian**



How to store data words in memory?

For example, how to store 32-bit quantity, $90AB12CD_{16}$, in memory:

Address	Value
1000	90
1001	AB
1002	12
1003	CD

Big Endian

Address	Value
1000	CD
1001	12
1002	AB
1003	90

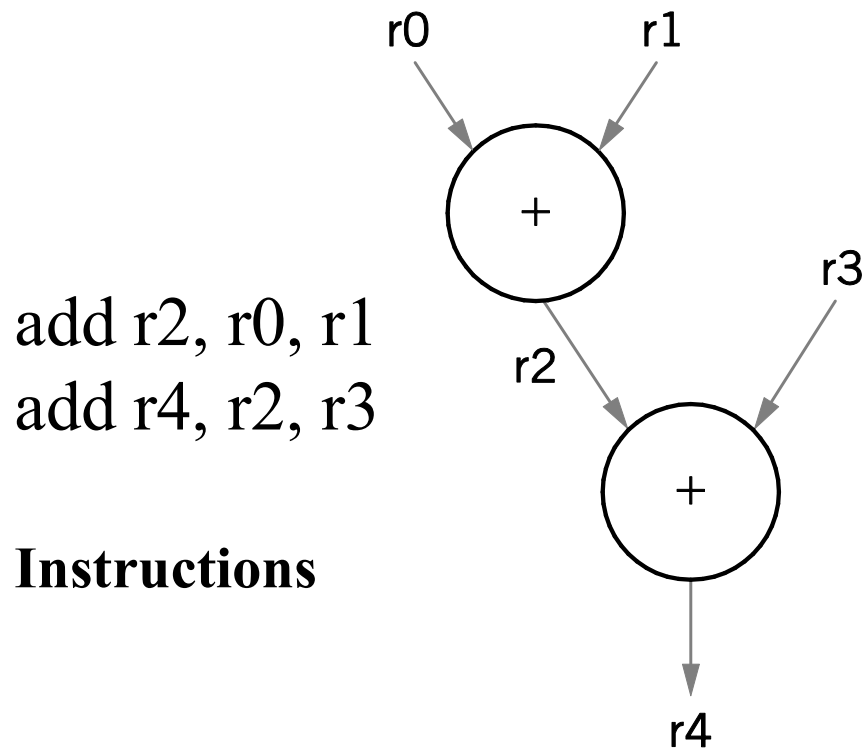
Little Endian

Computer architecture taxonomy

- **High-performance computer**
 - More than one instruction can be executed in a cycle
 - Condition: **adjacent instructions must not directly data/control depend on each other.**
 - Techniques:
 - Pipelined execution:
 - Multiple issues slots (e.g. Processing unit): computational resources on which an instruction can be executed, typically one per cycle

Data and control dependencies

Inter-instruction dependencies



`add r2, r0, r1`
`add r4, r2, r3`

Instructions

Data dependencies

- `r0 + r1` must be performed before `r2 + r3`.

`bnz r3, foo`
`add r0, r1, r2`
Foo: ...

Control dependencies

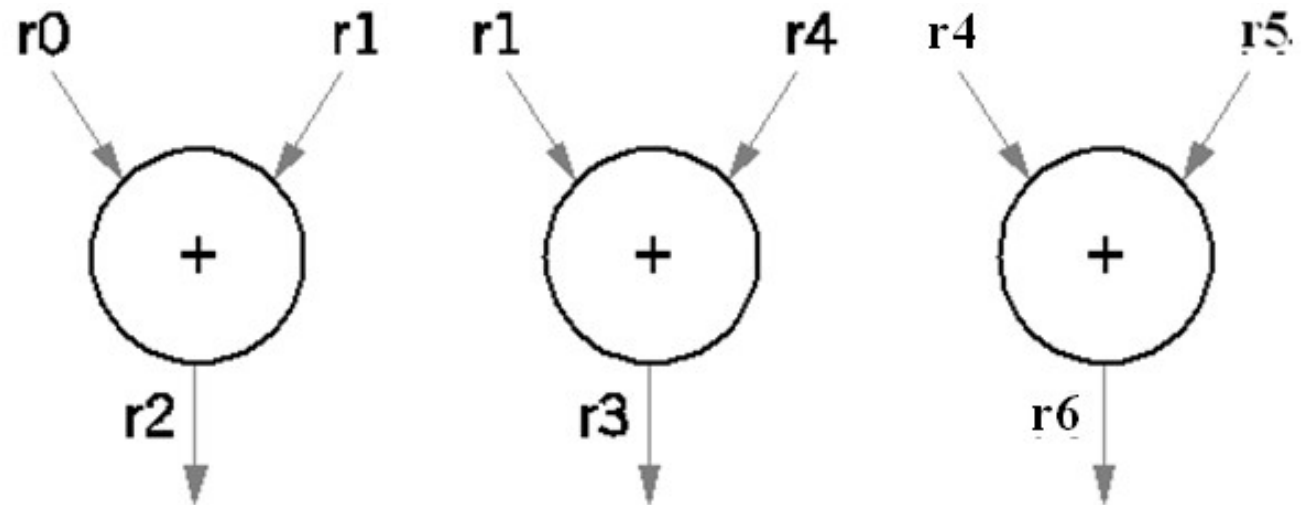
- "Add" will be executed only if `bnz` fails.

Data dependencies and parallelism

Inter-instruction dependencies

add r2, r0, r1
add r3, r1, r4
add r6, r5, r4

Instructions

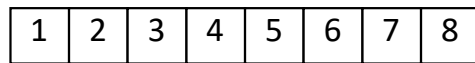
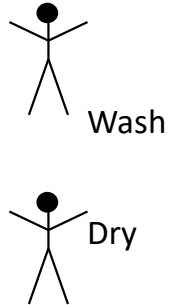


- All three additions can be performed at the same time.

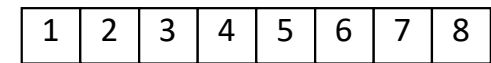
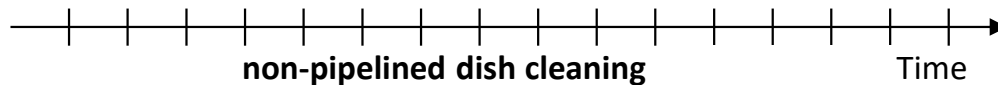
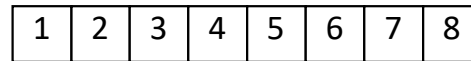
Superscalar and VLIW Architectures

- Performance can be improved by:
 - Faster clock (but there's a limit)
 - Pipelining: slice up instruction into stages, overlap stages
 - Multiple *ALUs* to support more than one instruction stream
 - Superscalar
 - Scalar: non-vector operations
 - Fetches instructions in batches, executes as many as possible
 - » May require extensive hardware to detect independent instructions
 - VLIW: each word in memory has multiple independent instructions
 - » Relies on the compiler to detect and schedule instructions
 - » Currently growing in popularity

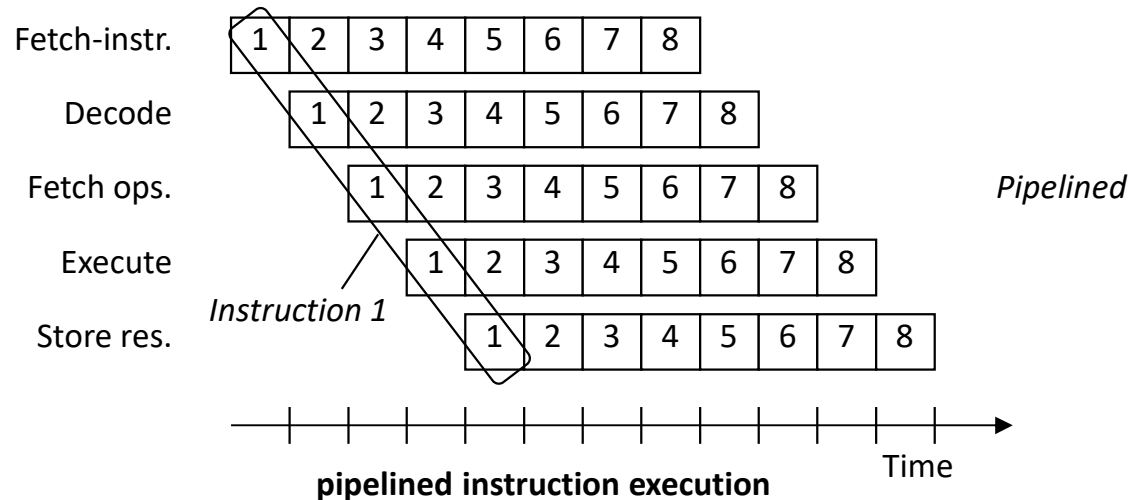
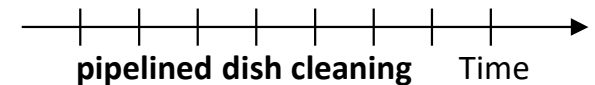
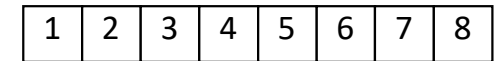
Pipelining: Increasing Instruction *Throughput*



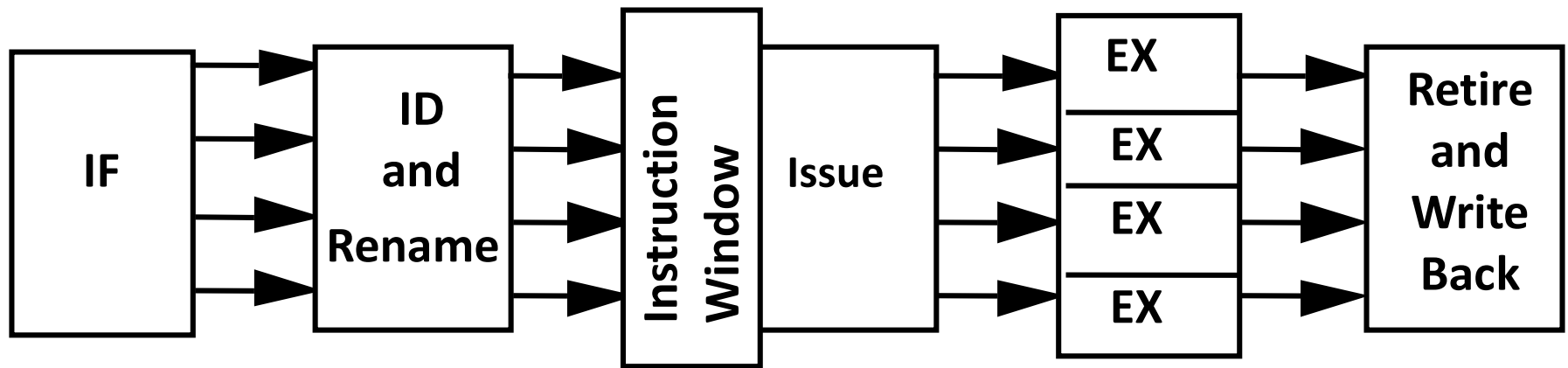
Non-pipelined



Pipelined



Multiple ALUs' Pipeline



- Instructions in the instruction window are **free from control dependencies** due to branch prediction, and **free from name dependencies** due to register renaming.

Computer architecture taxonomy

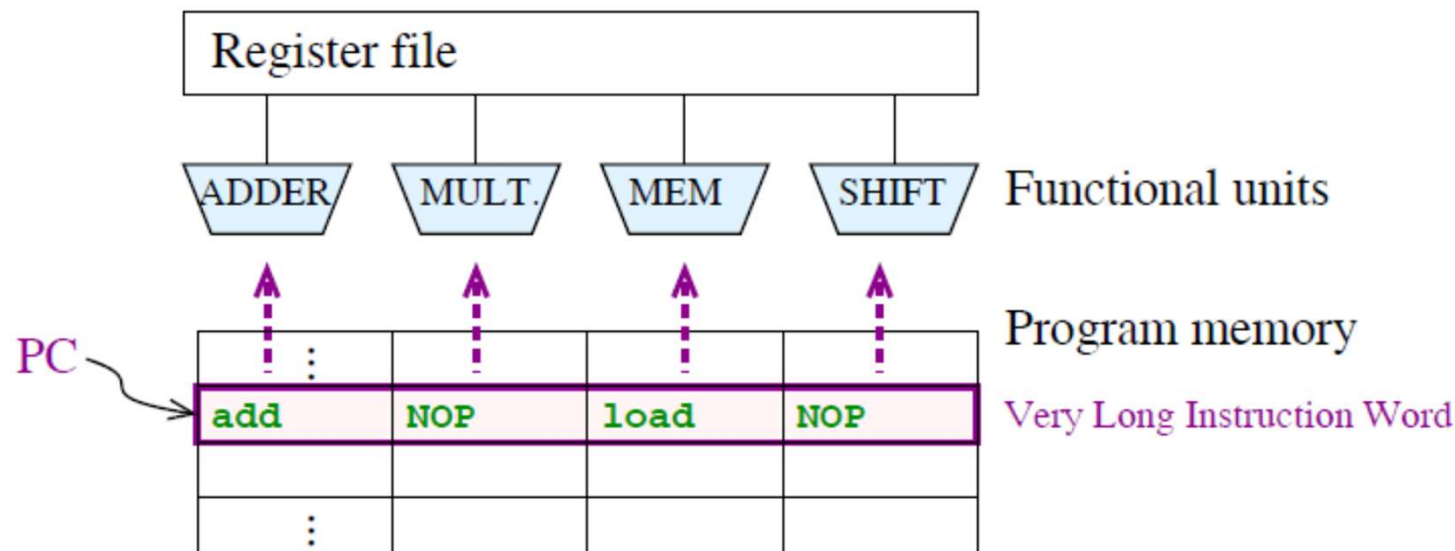
Out-of-order superscalar processor

- Performs several instructions simultaneously.
 - use hardware to analyze the instruction stream during execution to find sets of instructions that can be executed together.
- Application: Desktop and laptop computers

Computer architecture taxonomy

VLIW: very long instruction word

- Performs several instructions simultaneously.
 - Architecture usually restricts the combination of instructions that can be performed at once (i.e. issue packet).
 - rely on the compiler to identify sets of instructions that can be executed in parallel.
- Applications: Digital signal processing systems



A traditional VLIW processor with very long instruction words consisting of four issue slots, each one controlling one functional unit.

Computer architecture taxonomy

Superscalar vs. VLIW

Superscalar	VLIW
Superscalar runs standard code, determines parallel operations at run time by hardware	VLIW determines parallelism at compile time
superscalar processors are more expensive in both cost and energy consumption	advanced compiler technology must be available together with VLIW technology

Outline

- Basic Processor Architecture
- Computer architecture taxonomy
- **Basic concept of Instruction set**
- ARM instruction set
 - *Case study*: ARM Cortex-M0 architecture
- Summary



Instruction Set

- **The **Instruction Set** defines:**
 - the interface between software modules and the underlying hardware;
 - what the hardware will do under certain circumstances.
- **Characteristics:**
 - fixed versus variable length;
 - addressing modes;
 - numbers of operands;
 - types of operations supported

Instruction Set

- Why do we need to know the instruction set?
 - Most software design effort for many systems is possible to be implemented in high-level languages and knowledge of the instruction set is not required
- But.....
 - Embedded systems require initialisation code and interrupt routines
 - the instruction set is the key to analyzing the performance of programs
 - All systems require debugging – possibly at the instruction level
 - Performance gains can be made by writing assembler routines
 - Some features of the processor architecture are not available with compilers
 - Etc.

Programming model

- Programming model: registers visible to the programmer.
 - Some registers are not visible (IR).

Assembly language

- Textual description of instructions, instead of their binary representation.
- Basic features:
 - One instruction per line.
 - Labels provide names for addresses (usually in first column).
 - Instructions often start in later columns.
 - Columns run to end of line.

ARM assembly language example

```
label1  ADR  r4, c
        LDR  r0, [r4] ; a comment
        ADR  r4, d
        LDR  r1, [r4]
        SUB  r0, r0, r1 ; comment
```

Assembly-Level Instructions

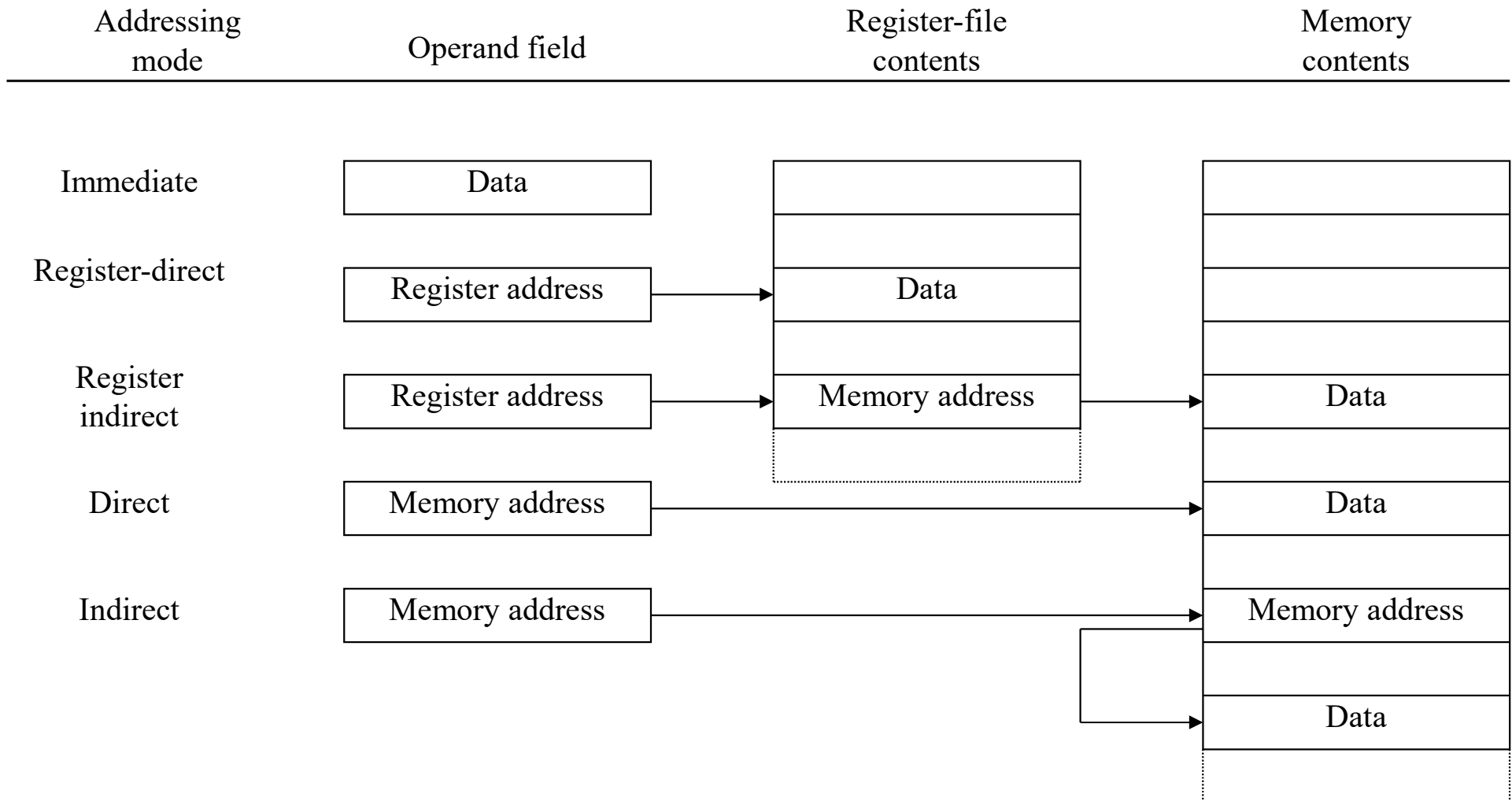
Instruction 1	opcode	operand1	operand2
Instruction 2	opcode	operand1	operand2
Instruction 3	opcode	operand1	operand2
Instruction 4	opcode	operand1	operand2
...			

- Instruction Set
 - Defines the legal set of instructions for that processor
 - Data transfer: memory/register, register/register, I/O, etc.
 - Arithmetic/logical: move register through ALU and back
 - Branches: determine next PC value when not just PC+1

A Simple (Trivial) Instruction Set

Assembly instruct.	First byte		Second byte	Operation
MOV Rn, direct	0000	Rn	direct	$Rn = M(\text{direct})$
MOV direct, Rn	0001	Rn	direct	$M(\text{direct}) = Rn$
MOV @Rn, Rm	0010	Rn	Rm	$M(Rn) = Rm$
MOV Rn, #immed.	0011	Rn	immediate	$Rn = \text{immediate}$
ADD Rn, Rm	0100	Rn	Rm	$Rn = Rn + Rm$
SUB Rn, Rm	0101	Rn	Rm	$Rn = Rn - Rm$
JZ Rn, relative	0110	Rn	relative	$PC = PC + \text{relative}$ (only if Rn is 0)
	opcode		operands	

Addressing Modes



Sample Programs

C program

```
int total = 0;
for (int i=10; i!=0; i--)
    total += i;
// next instructions...
```

Equivalent assembly program

```
0    MOV R0, #0;      // total = 0
1    MOV R1, #10;     // i = 10
2    MOV R2, #1;      // constant 1
3    MOV R3, #0;      // constant 0

Loop: JZ R1, Next;     // Done if i=0
5    ADD R0, R1;       // total += i
6    SUB R1, R2;       // i--
7    JZ R3, Loop;      // Jump always

Next: // next instructions...
```

- Exercise: Try some others
 - Handshake: Wait until the value of M[254] is not 0, set M[255] to 1, wait until M[254] is 0, set M[255] to 0 (assume those locations are ports).
 - (Harder) Count the occurrences of zero in an array stored in memory locations 100 through 199.